Apply, Don't Articulate: A Multi-Paradigm, Multi-Scale Study of Internal-Model Prompting in LLMs

Mohamed A. Mabrok
m.a.mabrok@gmail.com

April 2026

Abstract

The Internal Model Principle (IMP) of Francis and Wonham [2] states that any feedback controller that achieves perfect asymptotic tracking of a class of exogenous signals must reproduce, as an internal subsystem, the dynamics that generate those signals. A practitioner reading of this principle, popular in agentic LLM design, prescribes that prompting a large language model to *articulate* an internal model of its environment before acting should improve closed-loop performance, increasingly so with scale. We test this prescription in two distinct symbolic paradigms: deterministic integer sequence prediction (a discovery benchmark of 30 hand-curated tasks plus a confirmation benchmark of 260 programmatically-generated tasks with multi-seed sampled decoding) and code debugging (30 buggy Python functions with hidden test-suite grading). We apply both benchmarks across up to five model scales (Qwen2.5-7B, 14B, 32B; Llama3.1-70B 4-bit; Qwen2.5-Coder-7B, 14B, 32B), for a total of over 10,000 prompted trials. Across this corpus, four findings stand together. *First*, articulating an internal model never significantly helps in either paradigm, at any scale, under any of three articulation phrasings tested. *Second*, the IM-Hypothesis prompt significantly hurts at 22 of 24 difficulty × scale × phrasing cells in the sequence-prediction confirmation benchmark, and produces a 97**-percentage-point pass-rate collapse** at Qwen2.5-Coder-14B on the code-debugging benchmark ($t = 25$–42, $p < 10^{-20}$ at every difficulty band) – the largest single prompt-induced regression observed in the study. *Third*, the harm exhibits a scale-dependent inverted-U in both paradigms, peaking at 14–32B and weakening at the smallest and largest tested scales. *Fourth*, four causal-mechanism arms (self-consistency, oracle, hypothesis-first, equal-budget), applied in both paradigms, refute the simplest tokenized-commitment account but support a sharper one: *the harm is in rule-generation, not rule-application.* Self-consistency over 5 sampled diagnoses does not recover the reactive baseline at the catastrophe scale; reordering the prefix does not erase the harm; equal-budget is inert. But *providing* the rule directly (the oracle arm) helps significantly with growing magnitude on hard tasks (-0.84 log-error gap at Qwen2.5-32B on hard sequences; $+0.10$ pass-rate gain at Qwen2.5-Coder-32B on hard code). These results yield a clean deployment recommendation that survives across two paradigms and seven model scales: *do not ask the LLM to articulate its internal model; provide the model and ask it to apply.* The benchmarks, prompts, graders, and analysis pipeline are released.

Keywords: internal model principle, large language models, sequence prediction, code debugging, prompt engineering, chain-of-thought, control theory, scaling, negative result, reproducibility.

1 Introduction

LLMs are increasingly deployed as *controllers*: agents that maintain context, plan over horizons, and produce actions in response to a stream of observations. This usage invites a question that

has a sharp, classical answer for linear feedback systems but no settled answer for neural networks: *what must a controller contain in order to track an exogenous reference?* For LTI systems, Francis and Wonham [2] answered with the Internal Model Principle (IMP): any controller that achieves perfect asymptotic tracking of a class of disturbance or reference signals must reproduce, internally, the dynamics that generate those signals. The IMP is an architectural necessity; it is silent about *representation.* A model can live in a state-space realization, in network weights, in activations, or in tokens.

A widespread practitioner translation of the IMP to LLM agents asserts that the representation is tokens: *ask the model to write down its hypothesis about the environment before acting, and performance will improve.* This translation is operational — it is implemented by editing a system prompt — and it is the implicit logic behind a substantial subset of agentic-system design templates as well as the well-studied chain-of-thought (CoT) prompting paradigm [6, 3, 5]. The expectation is straightforward: articulating an internal model should help, increasingly so with scale, and most of all on tasks that genuinely require model-based prediction.

This paper tests that expectation directly and reports a negative result. We construct a clean sequence-prediction game in which a hidden generator $f\colon \mathbb{N} \to \mathbb{Z}$ produces an integer target at each step k, the LLM observes the history, emits a prediction $\hat{y}_k$, and the true y_k is appended. We compare three matched system prompts: a Reactive baseline that asks for one integer with no scaffolding, an im-Hyp prompt that explicitly invokes the IMP and instructs the model to state a one-line hypothesis about $f(k)$ before predicting, and an im-CoT zero-shot CoT variant.

We then run two benchmarks against the same three prompts on four open-weight model scales spanning roughly an order of magnitude. A *discovery* benchmark, with 30 hand-curated tasks, single-seed greedy decoding, and a paired-t test over 10 tasks per difficulty band, is used to identify candidate effects. A *confirmation* benchmark, with 260 programmatically generated tasks across 30 parametric families, multi-seed sampled decoding at temperature 0.7, and cluster-bootstrap confidence intervals that resample tasks (the dominant variance cluster), is used to test whether the candidate effects survive at the level of statistical strength a careful reviewer requires. Together the two benchmarks comprise 5,760 prompted trajectories.

The results, summarized in Sections 6 and 7, support three claims:

1. *Articulating an internal model never significantly helps.* Across all 36 *difficulty* $\times$ *scale* $\times$ *phrasing* cells in the two benchmarks, no paired test rejects the null in the helping direction at $\alpha = 0.05$.

2. *The harm is robust at every scale.* In the confirmation benchmark, the im-Hyp prompt significantly degrades performance on 22 of 24 cells (paired t-tests on log-error, $p < 10^{-90}$ at the smallest scale and $p < 10^{-6}$ at the largest). Cluster-bootstrap CIs that resample tasks exclude zero on the medium-difficulty band at 7B, 14B and 32B.

3. *Chain-of-Thought articulation has an inverted-U harm profile in scale.* The mean log-harm of im-CoT on hard tasks is $+0.02, +1.38, +1.60, +0.64$ at 7B, 14B, 32B, 70B respectively, peaking in the middle. We propose, and discuss, a tokenized-commitment mechanism that predicts this shape: a too-small model writes inert reasoning that the integer-emission step ignores, an intermediate model writes *competent-looking* but wrong reasoning that the integer-emission step computes from, and a largest-tested model partially recovers by recognising and correcting the chain.

A representative single-cell failure illustrates the mechanism: on the Fibonacci task at Qwen-7B, the Reactive condition attains zero asymptotic error while the im-Hyp condition attains an asymptotic error of 71.17. Inspection of the raw replies reveals that the im-Hyp replies emit an incorrect recurrence on the `HYPOTHESIS:` line — typically a wrong linear extrapolation or a wrong arithmetic recurrence — and then deterministically compute integer predictions forward from the wrong rule, accumulating error linearly in k. The Reactive condition, by

contrast, emits the correct integer through pure pattern completion. The same phenomenon recurs at every scale.

We are careful to be precise about what these results refute and what they do not. They refute the prompting reading of the IMP – the operational claim that asking an LLM to externalise an internal model improves prediction. They do not refute the IMP itself, which is a theorem about representations available to a feedback controller, not about prompts. Section 2 works through this distinction in detail. Sections 12 and 13 state the experimental scope honestly and list pre-registered extensions.

Contributions. (i) A principled translation of the IMP to LLM-agent prompting, together with three reasons it is not entitled to the strength of the original theorem. (ii) A reproducible 30-task discovery benchmark and a 260-task confirmation benchmark with 30 parametric task families, deterministic given a sampling seed. (iii) A multi-seed sampled-decoding protocol with cluster-bootstrap CIs that resample tasks, addressing the standard "small-n sensitivity" reviewer concern. (iv) A robust, multi-scale negative result with an explicit tokenized-commitment mechanism, including a previously unreported inverted-U harm pattern for CoT in scale. (v) Code, prompts, and analysis pipeline released at `https://github.com/Minds-R-Lab/phase_margin_llm`.

2 The Internal Model Principle and Its Prompting Analogue

We restate the classical IMP precisely so the prompting analogue remains under tight scrutiny.

The classical principle. Consider an LTI plant P and controller K in standard unity feedback, with reference r, output y, and tracking error $e = r - y$. Suppose r belongs to a class generated by an exosystem with state-space realization (A_e, B_e, C_e) – e.g., A_e has poles at $\{0, \pm j\omega_1, \pm j\omega_2, \dots\}$ for steps and persistent sinusoids. Francis and Wonham [2, Theorem 1] establishes: *If the closed loop is internally stable and asymptotically rejects the disturbance class generated by* (A_e, B_e, C_e)*, then the dynamic modes of the exosystem must be reproduced inside the loop.* This is a necessity result – internal stability plus asymptotic tracking implies the internal model.

The prompting analogue. Translated to LLM agents, the analogue reads:

> *If the agent must achieve low asymptotic error in tracking a class of structured signals, the agent must, in some representation, contain a model of those signals; therefore, prompting the agent to externalize and use that model in tokens should help.*

Three observations show this analogue is not entitled to the strength of the original.

Modes, not language. The classical theorem is about which poles appear in the loop in a fixed representation (state-space, Laplace). Translating "contains the model" to "writes the model in words" requires that the textual articulation be representation- preserving. This is a strong assumption. An LLM's implicit model of integer sequences lives in network weights and activations; articulating that implicit model in tokens may be lossy or distortive.

Asymptotic exact tracking, not approximate prediction. The classical result is sharp at zero steady-state error. An agent doing finite-horizon prediction with average error as the loss function is not in the regime where the sharp condition is operative, and does not inherit the necessity-direction conclusion *a priori*.

"Contains" is not "commits to". The signal class admits multiple realisations of its internal model. A classical controller may carry several modes simultaneously and combine them linearly. An LLM that has just emitted `HYPOTHESIS: f(k) = 5k + 1` has, in the next-token sense, *committed* to one realisation: subsequent integer predictions will be conditioned on that text and the autoregressive decoding will lock in the implied computation.

These caveats predict that the prompting analogue is fragile. The empirical question – whether the analogue nevertheless yields a useful heuristic – is what the rest of this paper answers.

3 Related Work

Chain-of-thought prompting. Wei et al. [6] introduced chain-of-thought (CoT) prompting and demonstrated that prompting LLMs to produce intermediate reasoning steps before answering improves accuracy on arithmetic, commonsense, and symbolic-reasoning tasks, especially for the largest models in their original setup ($\geq$ 100B parameters). Kojima et al. [3] simplified this to a zero-shot "Let's think step by step" prompt, the exact phrasing we use in our IM-CoT condition. Wang et al. [5] showed self-consistency over multiple sampled chains improves CoT accuracy further. More recently, the meta-analysis of Sprague et al. [4] surveyed 100+ CoT papers and ran controlled evaluations on 20 datasets; their headline finding is that CoT helps mainly on math and symbolic-execution tasks, with much smaller gains elsewhere. Our paradigm – short, deterministic integer-sequence prediction – is symbolic in nature, the regime where CoT is supposed to help. That *both* the IMP-flavoured IM-HYP and the "Let's-think-step-by-step" IM-CoT prompts *hurt* on this paradigm sharpens the negative-effects-of-CoT picture.

Multi-agent LLM failures. At a system level, Cemri et al. [1] introduce MAST – a taxonomy of 14 failure modes across 7 multi-agent frameworks built from expert annotation of 1,600+ traces. Their Category I (system design issues) includes failures driven by prompt structure and role specification. Our finding contributes a single-agent specialisation: a system prompt that asks for an articulated internal model can drive performance down on a task family that the same model would solve with no scaffolding.

Control-theoretic perspectives on neural systems. A growing line of work imports control-theoretic concepts – stability, observability, feedback – into the analysis of neural networks and LLM agents. We take the opposite stance from much of this work: rather than borrowing a guarantee and asserting the LLM inherits it, we test, empirically and conservatively, whether one of the most foundational such guarantees translates at the prompting level. It does not. This contributes an instructive data point about which control-theoretic results have meaningful prompt-level analogues.

Negative results with scale. Recent literature has documented several scale-dependent effects in LLMs that contradict naive scaling intuitions, including inverse-scaling failures (where larger models do worse at certain tasks) and U-shaped scaling. Our inverted-U harm pattern for CoT (Section 7.4) is consistent with that broader picture: the harm peaks where the model is competent enough to write articulate but wrong reasoning, but not yet competent enough to recognise and correct it.

4 Benchmark Design and Methodology

We describe two benchmarks. Benchmark v1 is a small, hand-curated discovery benchmark; benchmark v2 is a large, programmatically generated, multi-seed confirmation benchmark. The same three system prompts and the same evaluation metric are used in both benchmarks; only the task pool, decoding configuration, and statistical analysis differ.

4.1 System prompts (verbatim)

The central finding of this paper depends on subtle differences between three system prompts. We give them verbatim below; they are identical across both benchmarks.

Reactive (no-scaffold baseline).

```
You are a sequence-prediction agent.  At each step you will be
shown all observations so far in the form
‘k=0: y=...; k=1: y=...; ...‘ and asked to predict the next value
y_k.  Reply with EXACTLY one integer on a line by itself.  No
commentary, no labels, no quotes.
```

im-Hyp (the canonical IMP-as-prompt).

```
You are a sequence-prediction agent operating under the Internal
Model Principle.  At each step you will be shown all observations
so far and asked to predict the next value y_k.  You MUST first
state a one-line HYPOTHESIS about the function f(k) that generated
the observed values, then on a NEW LINE write your single integer
prediction for y_k.

Reply format (exactly two lines):
    HYPOTHESIS: <your one-line hypothesis>
    <integer prediction>
```

im-CoT (zero-shot chain-of-thought).

```
You are a sequence-prediction agent.  At each step you will be
shown all observations so far and asked to predict the next value
y_k.  Let’s think step by step: briefly reason about what rule
could be generating these values and why your candidate next
value is consistent with it.  After your reasoning, write your
single integer prediction for y_k as the LAST line of your reply,
by itself, with no labels or punctuation.  The integer on the
last line will be parsed automatically.
```

The user prompt is identical across the three conditions:

```
Observations so far: k=0: y=...; k=1: y=...; ...

Predict y_k.
```

REACTIVE uses `max_tokens = 96`; IM-HYP uses `max_tokens = 96`; IM-COT uses `max_tokens = 256` (reasoning is longer). This is the only systematic difference in decoding configuration between conditions and we revisit it in Section 12.

4.2 Common evaluation metric

Each cell consists of a K-step trajectory. At step k, the model is shown the history of true observations $\{(j, y_j)\}_{j<k}$ and asked to predict y_k. The history is then advanced by appending the *true* y_k (not the prediction), so the loop is open in the prediction direction. The asymptotic error of the cell is the mean absolute error over the last ASYM steps:

$$E_{\text{asym}} = \frac{1}{\text{ASYM}} \sum_{k=K-\text{ASYM}}^{K-1} \left|\hat{y}_k - y_k\right|. \tag{1}$$

For statistical comparisons we work with $\log(E_{\text{asym}} + \varepsilon)$ with $\varepsilon = 0.5$ to keep the metric defined when one or both conditions score zero.

4.3 Benchmark v1: 30-task discovery benchmark

The discovery benchmark contains 30 deterministic, integer-valued generators, balanced across three difficulty bands of 10 tasks each. Difficulty was hand-assigned: *easy* tasks are those on which a no-reasoning baseline (echo recent observation) tends to be approximately correct; *medium* tasks require explicit pattern recognition but only one symbolic step to compute the next value; *hard* tasks require non-trivial computation even given a correct hypothesis. Trajectory length is $K = 14$ with $\textsc{asym} = 6$, except for one geometric task capped at $K = 12$ to avoid integer overflow in the prompt. Decoding is greedy (temperature = 0, single seed). The full task list and the exact generators are given in Table **??** of Appendix A.

The discovery benchmark uses paired t-tests on log-error per (band, IM-condition) pair, with pairing over the 10 tasks. $n = 10$ per test.

4.4 Benchmark v2: 260-task confirmation benchmark

A reviewer of the v1 benchmark observes, correctly, that single-seed greedy decoding gives no within-cell variance and that $n = 10$ tasks per band is sensitive to the specific tasks chosen. Benchmark v2 is engineered to address both prongs of this critique simultaneously.

Programmatic task families. Each difficulty band is a pool drawn from 10 parametric generator families. For example, the *easy* band families are: a constant integer, a positive linear function $ak + b$, a negative-slope linear function, a period-2 cycle, a period-3 cycle, two period-4 cycle variants, $k \bmod n$, an arithmetic progression ak, and an arithmetic progression with offset. The *medium* families include quadratic, quadratic with offset, negative quadratic, period-5 and period-6 cycles, piecewise linear, zigzag $k(-1)^k s$, $\lfloor k/n \rfloor$, triangle wave, and linear-plus-period. The *hard* families include Fibonacci with random initial conditions, capped geometric $c \cdot 2^k$, $k(k-1)$, triangle numbers, random digit cycles, $k^3 \bmod n$, conditional rules, regime-switching cycles, sums of two coprime periods, and general quadratics with mixed-sign coefficients. Each family is parameterized by integer ranges chosen to keep prompts bounded (target absolute value capped at 50,000) and to be distinguishable from one another.

A task is generated by drawing a parameter tuple from a family. The full task pool size is configurable; we use 100 tasks per band on the smallest model, scaling down to 20 on the largest to fit overnight wall-time. The set is deterministic given a sampling seed: the same `--task-seed` produces the identical task pool across machines. All four model runs in this paper use `--task-seed=2026`.

Multi-seed sampled decoding. Each cell is run with multiple LLM seeds at `temperature = 0.7`. The number of seeds per cell is 5 on the two smallest models, 3 on the two larger models, again to fit wall-time. Pairing is over (task, seed), so each (band, IM-condition) test has up to 500 paired observations rather than 10. Trajectory length is $K = 12$ with $\textsc{asym} = 5$ throughout.

Statistical analysis. For each (scale, band, IM-condition) triple we report:

- a paired t-test on log-error over (task, seed),
- a cluster-bootstrap 95% CI on the median log-difference, resampling at the *task* level (the dominant variance cluster), with 5,000 bootstrap replicates. Resampling tasks — not seeds — is what addresses the "sensitive to which tasks?" reviewer concern directly,
- a within-task vs. across-task variance decomposition.

Implementation. The benchmark writes per-cell results incrementally to a JSON-Lines file. An interrupted overnight run is resumed by re-invoking the same command; cells already present in the JSONL are skipped. Total wall-time across the four scales on a single H100 is approximately 26 hours.

4.5 Models

We test four open-weight model scales served via Ollama on a single NVIDIA H100 80GB:

- **Qwen2.5-7B-Instruct** (7.6B parameters)
- **Qwen2.5-14B-Instruct** (14.7B parameters)
- **Qwen2.5-32B-Instruct** (32.5B parameters)
- **Llama3.1-70B-Instruct, q4_K_M** (70B parameters, 4-bit quantized)

The four scales span roughly an order of magnitude in parameter count. All four are open-weight; the 70B is the only quantized model in the set. Frontier-scale closed models (Claude, GPT, Gemini) are not tested here; see Section 13.

5 Pre-registered Hypotheses

The naive prompting reading of the IMP, combined with the chain-of-thought literature, suggests three hypotheses. We state them before reporting results.

H1. IMP prompts hurt small models on tasks they can already solve reactively, but the harm decays monotonically with scale.

H2. IMP prompts help small models on hard tasks (replicating the chain-of-thought literature for math/symbolic reasoning).

H3. The joint behaviour shows a significant *difficulty* $\times$ *condition* $\times$ *scale* interaction.

The empirical findings, reported next, refine H1 (the harm decays in some bands and *migrates* in others), refute H2 (no help emerges anywhere, at any scale), and confirm a modified form of H3 (the interaction is present but in the harm-migration direction, and a previously unreported inverted-U appears for CoT).

6 Results: Discovery Benchmark

6.1 Main table

Table 1 reports the mean asymptotic error per (scale, difficulty, condition) cell on the v1 benchmark.

The discovery benchmark identifies the headline pattern: at every scale, on the medium difficulty band, IM-HYP significantly hurts (p values $0.001, 0.010, 0.043, 0.025$ across the four scales, combining via Stouffer's method to $Z = 5.06$, $p < 10^{-6}$). On the easy band, the harm is significant at the smaller scales and ties at the floor for the largest. On the hard band, the v1 benchmark is underpowered; significant effects appear only on the 32B-CoT cell.

6.2 Cross-scale trend on the easy band

The mean log-error gap of IM-HYP vs. REACTIVE on the easy band shrinks $+1.41 \to +1.08 \to +0.51 \to 0.00$ across the four scales. Treating the four points as a function of log(parameter count) gives Spearman $\rho = -1.00$ (perfect monotone decay) and Pearson $r = -0.998$ ($p = 0.002$ two-sided). The medium band gives a directionally consistent but not-quite-significant decay (Spearman -0.80, Pearson -0.92). The pattern is interpretable as the easy-band reactive baseline saturating at the floor of zero error, leaving no room for the prompt to harm; it does not generalize cleanly to the medium band, where reactive performance still has slack at every scale.

Table 1: Discovery benchmark (v1, $n = 10$ tasks per cell, single seed, greedy decoding). Mean asymptotic error per (scale, difficulty, condition). Lower is better. Bold marks cells where a paired t-test on log-error against the same row's REACTIVE baseline is significant at $p < 0.05$ in the *hurts* direction. No cell shows the help direction at any significance level.

Model	Difficulty	REACTIVE	IM-HYP	IM-COT
Qwen2.5-7B	easy	0.15	**6.08**†	0.57
	medium	1.58	**5.22**†	1.90
	hard	5.58	30.03	7.05
Qwen2.5-14B	easy	0.00	**1.72**†	**1.32**†
	medium	0.43	**9.22**†	**2.57**†
	hard	5.40	5.43	45.12
Qwen2.5-32B	easy	0.00	0.75	0.25
	medium	1.73	**2.48**†	2.30
	hard	5.57	20.85	**13.53**†
Llama3.1-70B-q4	easy	0.00	0.00 (tied)	0.02
	medium	0.37	**0.97**†	**1.83**†
	hard	6.77	8.03	11.25

Table 2: Catastrophic single-cell failures (v1). Cells where REACTIVE scores at floor and the articulated condition does not. The phenomenon recurs at every scale, including the 32B model.

Model	Task	REACTIVE	IM-HYP	IM-COT
Qwen2.5-7B	`e04_linear_5k`	0.00	41.67	0.00
Qwen2.5-7B	`h01_fibonacci`	0.00	71.17	0.00
Qwen2.5-7B	`h02_geometric_2`	0.00	160.00	0.00
Qwen2.5-14B	`m02_quadratic_offset`	0.00	66.17	7.83
Qwen2.5-14B	`h02_geometric_2`	0.00	0.00	384.00
Qwen2.5-32B	`m02_quadratic_offset`	0.00	8.83	16.50
Qwen2.5-32B	`h02_geometric_2`	0.00	159.50	0.00
Qwen2.5-32B	`h03_squares_minus_k`	0.00	0.00	68.67

6.3 Catastrophic single-cell failures

A subset of cells exhibit a particularly informative failure mode: REACTIVE achieves zero asymptotic error and the articulated condition produces a persistent error of tens to hundreds of units. Such cells rule out the explanation "the model cannot do the task." Table 2 lists the most striking cases at each scale. A representative example: on Fibonacci with Qwen-7B, REACTIVE scores 0.00 while IM-HYP scores 71.17. Inspection of the raw replies shows the model emits a wrong recurrence on the `HYPOTHESIS:` line and computes integer predictions deterministically forward from it. We return to this mechanism in Section 8.

7 Results: Confirmation Benchmark

7.1 Main table

Benchmark v2 increases n per cell from 10 to between 60 and 500 paired observations, replaces hand-curated tasks with programmatically sampled tasks, and introduces multi-seed sampled decoding. The mean asymptotic errors are reported in Table 3.

Table 3: Confirmation benchmark (v2, programmatic tasks, multi-seed sampled decoding at $T = 0.7$). Mean asymptotic error per (scale, difficulty, condition). Sample size per condition, N_{cells}, is given in the second column. Across this corpus, 22 of 24 paired tests reject the null in the *hurts* direction at $\alpha = 0.05$; zero reject in the *helps* direction. Bold marks $p < 0.05$ hurt significance of the paired t-test.

Model	Difficulty	N_{cells}	REACTIVE	IM-HYP	IM-COT
Qwen2.5-7B	easy	500	~ 0.08	$\mathbf{2.6}^{\dagger}$	$\mathbf{0.95}^{\dagger}$
	medium	500	1.83	$\mathbf{6.67}^{\dagger}$	$\mathbf{1.94}^{\dagger}$
	hard	500	7.56	$\mathbf{72.93}^{\dagger}$	8.82
Qwen2.5-14B	easy	400	0.00	$\mathbf{1.60}^{\dagger}$	$\mathbf{1.23}^{\dagger}$
	medium	400	0.19	$\mathbf{4.12}^{\dagger}$	$\mathbf{2.52}^{\dagger}$
	hard	400	6.12	$\mathbf{11.90}^{\dagger}$	$\mathbf{21.78}^{\dagger}$
Qwen2.5-32B	easy	180	0.00	$\mathbf{1.30}^{\dagger}$	$\mathbf{0.46}^{\dagger}$
	medium	180	0.33	$\mathbf{4.63}^{\dagger}$	$\mathbf{6.34}^{\dagger}$
	hard	180	47.16	$\mathbf{45.80}^{\dagger}$	$\mathbf{29.24}^{\dagger}$
Llama3.1-70B-q4	easy	60	0.07	$\mathbf{0.19}^{\dagger}$	$\mathbf{1.12}^{\dagger}$
	medium	60	0.40	$\mathbf{1.32}^{\dagger}$	$\mathbf{2.14}^{\dagger}$
	hard	60	6.79	10.64	$\mathbf{17.67}^{\dagger}$

7.2 Paired t-tests

The paired t-statistics on log-error per cell, against the REACTIVE baseline, are reported in Table 4. Recall: positive t here means IM-HYP or IM-COT has *larger* log-error than REACTIVE, i.e., the IM condition hurts.

7.3 Cluster-bootstrap confidence intervals

A paired t-test pools across tasks; an alert reviewer asks how sensitive the conclusion is to the specific tasks sampled. Table 5 reports cluster-bootstrap 95% confidence intervals on the median log-difference, resampling at the task level with 5,000 replicates. An interval that excludes zero is robust to the specific task sample.

The most robust headline that survives both the parametric t-test and the non-parametric task-cluster bootstrap is: *the IM-Hypothesis prompt significantly hurts on the medium-difficulty band at* $7B$, $14B$, *and* $32B$, with bootstrap CIs $[+0.76, +1.29]$, $[+1.22, +1.95]$, and $[+0.37, +1.44]$ respectively. At 70B the bootstrap CI on medium-Hyp is $[0, +0.62]$, just touching zero, although the paired t-test is highly significant ($p < 10^{-6}$).

We emphasize the principled reading of the gap between the two tests. The paired t-test detects mean log-difference; the bootstrap CI on the median is dominated by tasks where both conditions tie. For IM-COT in particular, a large fraction of tasks have identical error under both conditions (CoT often reduces to reactive when the reasoning chain ends with the same integer); the median-of-bootstrap is therefore zero while the mean is positive. This is a feature of the task pool: harm is concentrated on a subset of tasks, not uniformly spread.

7.4 Inverted-U harm profile of CoT in scale

Reading the IM-COT column of Table 3 by scale on the hard band reveals a non-monotone pattern: mean log-harm $+0.02 \rightarrow +1.38 \rightarrow +1.60 \rightarrow +0.64$ at 7B, 14B, 32B, 70B respectively, peaking in the middle. The same shape appears on the medium band, less pronounced. Figure 6 plots the four points by scale on the hard and medium bands. The shape is consistent with a tokenized-commitment account:

Table 4: Confirmation benchmark paired t-tests on log-error $\log(E_{\text{asym}}+0.5)$. REACTIVE vs. each IM condition, by scale and difficulty. Positive t means the IM condition hurts. At $\alpha = 0.05$, 22 of 24 tests reject the null in the *hurts* direction; the only non-significant cells are the 7B-hard-CoT and 70B-hard-Hyp.

Model	Difficulty	vs. IM-HYP		vs. IM-COT	
		t	p	t	p
Qwen2.5-7B	easy	+27.0	$\mathbf{8.7{\cdot}10^{-100}}^{\dagger}$	+6.16	$\mathbf{1.5{\cdot}10^{-9}}^{\dagger}$
	medium	+23.2	$\mathbf{4.4{\cdot}10^{-81}}^{\dagger}$	+3.30	$\mathbf{0.0010}^{\dagger}$
	hard	+16.7	$\mathbf{7.2{\cdot}10^{-50}}^{\dagger}$	+0.50	0.62
Qwen2.5-14B	easy	+16.6	$\mathbf{2.3{\cdot}10^{-47}}^{\dagger}$	+15.4	$\mathbf{3.2{\cdot}10^{-42}}^{\dagger}$
	medium	+27.2	$\mathbf{5.6{\cdot}10^{-93}}^{\dagger}$	+19.8	$\mathbf{2.3{\cdot}10^{-61}}^{\dagger}$
	hard	+12.8	$\mathbf{1.3{\cdot}10^{-31}}^{\dagger}$	+15.6	$\mathbf{5.0{\cdot}10^{-43}}^{\dagger}$
Qwen2.5-32B	easy	+11.1	$\mathbf{3.1{\cdot}10^{-22}}^{\dagger}$	+6.57	$\mathbf{5.3{\cdot}10^{-10}}^{\dagger}$
	medium	+12.3	$\mathbf{1.8{\cdot}10^{-25}}^{\dagger}$	+9.29	$\mathbf{5.2{\cdot}10^{-17}}^{\dagger}$
	hard	+6.33	$\mathbf{1.9{\cdot}10^{-9}}^{\dagger}$	+9.95	$\mathbf{7.9{\cdot}10^{-19}}^{\dagger}$
Llama3.1-70B-q4	easy	+2.67	$\mathbf{0.0098}^{\dagger}$	+3.35	$\mathbf{0.0014}^{\dagger}$
	medium	+5.61	$\mathbf{5.7{\cdot}10^{-7}}^{\dagger}$	+6.28	$\mathbf{4.5{\cdot}10^{-8}}^{\dagger}$
	hard	+1.61	0.11	+3.15	$\mathbf{0.0026}^{\dagger}$

- The smallest model (7B) often fails to follow the CoT scaffolding tightly; its reasoning chain is short, generic, and ends in an integer largely independent of the chain's content. In effect, IM-COT collapses to REACTIVE for the smallest model, and the harm is small.
- The intermediate models (14B, 32B) write *competent-looking* reasoning chains – recognisably sequence-pattern-fitting prose – that often arrive at a wrong conclusion and pull the integer prediction with them. The harm is large.
- The largest model (70B-q4) writes longer, more careful reasoning chains that more often catch their own errors and re-converge to the correct integer. The harm is intermediate.

This profile is a positive empirical contribution that the v1 benchmark, with its small n, could not see.

7.5 Variance decomposition

Within-task variance and across-task variance of log-error are reported in Appendix Table **??**. The headline fact: the dominant variance source is across-task, not within-task, by factors of 5–20× on the REACTIVE and IM-HYP conditions. This implies that pairing by task – the v1 strategy – captured the right signal, and that multi-seed decoding contributes additional power but does not change the qualitative conclusion. An exception is IM-COT, where within-task variance grows substantially due to the diversity of reasoning chains across seeds; this is reflected in the conservative bootstrap CIs for the IM-COT column of Table 5.

8 Mechanism: Tokenized Commitment

Three independent observations point to the same mechanism for the harm.

Observation 1. Catastrophic single-cell failures (Tables 2 and **??**) all share the same structural form: the model emits a wrong rule on the `HYPOTHESIS:` line and then deterministically computes integer predictions forward from the wrong rule. The errors are not noisy – they are exact arithmetic consequences of the wrong hypothesis text. On Fibonacci with Qwen-7B,

Table 5: Cluster-bootstrap 95% CI on the median log-difference $\log(E^{\text{IM}}_{\text{asym}}+0.5)-\log(E^{\text{Reactive}}_{\text{asym}}+0.5)$, resampling at the task level (5,000 replicates). An interval excluding zero in the positive direction implies the IM condition robustly hurts even under task resampling. The bootstrap is markedly more conservative than the paired t-test because most tasks have median log-difference exactly zero (both conditions tied); the test detects the heavy tail of large-error tasks while the median-of-bootstrap stays at zero unless the tail is large relative to the task pool.

Model	Difficulty	vs. IM-Hyp (median, 95% CI)	vs. IM-CoT (median, 95% CI)
Qwen2.5-7B	easy	**1.44** [+1.10, +1.69]†	0.00 [0, 0]
	medium	**1.00** [+0.76, +1.29]†	0.00 [0, 0]
	hard	**0.71** [+0.40, +1.31]†	0.00 [0, 0]
Qwen2.5-14B	easy	0.34 [0, +0.96]	0.00 [0, +0.59]
	medium	**1.67** [+1.22, +1.95]†	**0.96** [+0.35, +1.25]†
	hard	**0.28** [+0.09, +0.41]†	**0.43** [+0.29, +0.95]†
Qwen2.5-32B	easy	0.17 [0, +0.96]	0.00 [0, 0]
	medium	**0.79** [+0.37, +1.44]†	0.34 [0, +1.22]
	hard	**0.32** [+0.08, +0.82]†	**0.46** [+0.11, +2.24]†
Llama3.1-70B-q4	easy	0.00 [0, 0]	0.00 [0, 0]
	medium	0.25 [0, +0.62]	**0.45** [+0.06, +1.19]†
	hard	0.00 [0, +0.18]	0.00 [0, +0.18]

Table 6: Inverted-U profile of IM-CoT harm in scale, on the hard and medium difficulty bands. Reported values are the mean log-difference of IM-CoT vs. Reactive on the v2 benchmark. Both bands peak at intermediate scale (14B–32B) and drop toward both extremes.

Difficulty band	Qwen-7B	Qwen-14B	Qwen-32B	Llama-70B-q4
hard	+0.02	+1.38	+1.60	+0.64
medium	+0.07	+1.03	+1.10	+0.75

the recurring wrong hypothesis is a linear extrapolation; on geometric-2 with Qwen-32B, the recurring wrong hypothesis is a wrong base. We provide raw replies for two cells in Appendix D.

Observation 2. The catastrophic failures recur at every scale tested in v1, including at 32B. This is inconsistent with the simplest "small models can't follow IMP scaffolding" explanation: a 32B model that demonstrates correct integer prediction under Reactive on a task and incorrect prediction under IM-Hyp on the same task is not failing for capability reasons.

Observation 3. The inverted-U in scale for IM-CoT fits the commitment account: CoT prompts only harm when the articulation is detailed enough to commit the decoder to a particular forward computation. At the smallest scale, articulation is sketchy and uncommitted; at the largest, articulation is corrigible. Maximum commitment occurs at intermediate scale.

Tokenized-commitment account. Within an autoregressive decoder, generated tokens are part of the prefix on which subsequent tokens are conditioned. The `HYPOTHESIS:` line is therefore a soft prefix on the integer-prediction line: once a particular rule is articulated, the decoder's next-token distribution over the integer line is conditioned on the rule. If the rule is correct, conditioning helps: the integer is the arithmetic image of the rule. If the rule is wrong, conditioning hurts: the integer is the arithmetic image of the wrong rule, and no further information helps recover from it. Crucially, Reactive never produces such a textual prefix. Its prediction is conditioned only on the history of integer pairs, not on a self-articulated rule. In effect, the implicit pattern-matcher encoded in the network's weights and activations is more accurate on this paradigm than the rule the model articulates as text.

In control-theoretic terms: the IM-Hypothesis prompt does not expose the implicit internal

model the network already has; it forces the decoder to *generate* a separate, articulated internal model in tokens, which then becomes the operative model for downstream prediction. In a fraction of cells, this articulated model is worse than the implicit one — and crucially, once articulated, the wrong model is committed to.

What the mechanism predicts. This account makes several testable predictions. Most directly, sampling several hypotheses per step and aggregating (self-consistency, in the sense of 5) should restore performance to near Reactive. Decomposing harm cell-by-cell should show that cells where the articulated hypothesis is correct match or beat Reactive, while cells where the hypothesis is wrong account for the entire im-Hyp−Reactive gap. The self-consistency prediction is pre-registered in Section 13; the hypothesis-correctness decomposition is reported next.

8.1 Direct test: hypothesis-correctness decomposition

We perform a free post-hoc analysis on the v2 cells. For every im-Hyp cell we extract the `HYPOTHESIS:` line at every step, parse it with a regex grammar of 20+ patterns covering linear, constant, $k \bmod n$, k^2, $(k \pm a)^2$, general quadratic, $k(k-1)$, triangle numbers $k(k+1)/2$, $k^3 \bmod n$, geometric $c \cdot 2^k$, triangle wave, $\lfloor k/n \rfloor$, zigzag, Fibonacci with explicit initial conditions, and several period/cycle phrasings (bracket lists, "alternates between A and B", "repeats every N terms with the pattern $A, B, \ldots$"). Each parsed rule is evaluated at the same step and compared to (a) the true target y_k and (b) the model's own integer prediction $\hat{y}_k$. We bin each cell on two binary axes over the asymptotic window: *rule-correct* (does the parsed rule predict y_k correctly on $\geq 66\%$ of asym-window steps?) and *rule-followed* (does the model's integer match the rule's prediction on $\geq 66\%$ of asym-window steps?). A cell where either rate falls in the borderline $(0.34, 0.66)$ region is binned "mixed"; cells where the parser cannot read any rule in the window are binned "unparseable".

Two failure modes are possible under the im-Hyp prompt:

1. **Wrong rule, committed** (*wrong_followed*): the model articulates an incorrect rule and emits the integer the rule predicts. Predicted by the original tokenized-commitment account.
2. **Correct rule, not applied** (*correct_not_followed*): the model articulates a correct rule but emits an integer that disagrees with it. This decoupling between rule-articulation and rule-application has no analogue under Reactive and is not predicted by the original account.

The grid is filled by *correct_followed* (commitment-faithful good case, expected to match Reactive) and *wrong_not_followed* (residual category).

Table 7 reports the 2×2 grid for all four scales.

The reading by scale is informative.

Llama-70B-q4: the original tokenized-commitment story holds sharply. When the parser confirms a correct rule *and* the model emits an integer consistent with it, the asymptotic error is at the reactive floor: 0.082 for IM vs. 0.045 for reactive, gap +0.04. Wrong-rule-followed cells (small n, but directionally consistent) carry the harm. Conditioned on the parser reading the rule, the entire im-Hyp−Reactive gap concentrates in cells where the articulated rule is wrong.

Qwen-14B: a second failure mode emerges. Correct-and-followed cells are nearly at floor (gap +0.88), but *correct_not_followed* cells carry a substantial +3.27 log-error gap. This is the *apply-step failure*: the articulated rule is right, the parser confirms it, the model just doesn't apply it. Reactive does not separate articulation from application and so does not exhibit this mode.

Qwen-7B: threshold artefact dominates. Even the *correct_followed* bin shows a +106 gap. This is not because correct articulation hurts; it is because at 7B the asym-window can

Table 7: Four-way (rule-correct × rule-followed) decomposition on v2 IM-HYP cells. Reported per (scale, bin): sample size n, mean asymptotic error of the IM cell, mean asymptotic error of the same task's reactive cell (paired by task), and the gap. Bins use a 66%-of-asym-window threshold on each axis; "mixed" is borderline on either axis; "unparseable" denotes cells where the regex grammar could not read a rule.

Model	Bin	n	mean IM	mean REACTIVE	gap
Qwen2.5-7B	correct_followed	34	106.51	0.25	+106.26
	correct_not_followed	95	11.34	0.21	+11.13
	wrong_followed	25	218.00	0.15	+217.85
	wrong_not_followed	129	22.33	5.21	+17.12
	mixed	139	8.02	0.55	+7.47
	unparseable	1078	26.53	3.74	+22.79
Qwen2.5-14B	correct_followed	81	0.91	0.03	+0.88
	correct_not_followed	48	3.37	0.10	**+3.27**
	wrong_followed	9	2.93	0.91	+2.03
	wrong_not_followed	132	3.21	1.05	+2.16
	mixed	105	3.02	0.22	+2.80
	unparseable	825	7.33	2.85	+4.48
Qwen2.5-32B	correct_followed	87	0.46	0.02	+0.45
	correct_not_followed	3	1.87	1.29	+0.58
	wrong_followed	6	3.97	2.07	+1.90
	wrong_not_followed	53	2.19	17.02	−14.82 ⋆
	mixed	58	6.52	0.71	+5.81
	unparseable	333	26.27	22.79	+3.48
Llama3.1-70B-q4	correct_followed	22	0.082	0.045	**+0.04**
	correct_not_followed	1	0.60	0.73	−0.13
	wrong_followed	2	4.00	2.63	+1.37
	wrong_not_followed	26	1.31	0.85	+0.45
	mixed	14	0.56	0.61	−0.05
	unparseable	115	5.89	3.46	+2.43

⋆ Negative gap is a within-task selection artefact on the heavy-tailed hard band (`h07_cubic_mod100` has reactive error ∼ 50); it is not evidence that IM-HYP helps.

include catastrophic arithmetic mistakes on the ≤ 33% of steps that the cell-level threshold did not require to be correct. The 4-way decomposition is therefore not cleanly diagnostic at this scale; what it shows is that per-step variability of integer emission overwhelms the rule-articulation distinction.

Coverage caveat. The parser is a regex grammar; it reads roughly 20–30% of cells across scales, with the remainder labelled "unparseable". The unparseable bin still shows a positive gap to reactive at every scale (gaps +22.8, +4.5, +3.5, +2.4 for 7/14/32/70B), consistent with the overall harm pattern. But these cells cannot be attributed to either of the two failure modes without additional analysis (e.g., LLM-as-classifier on the natural-language hypothesis content). The headline mechanism claim — that, conditioned on parseable rules, the harm concentrates in *correct_not_followed* (at 14B) and *wrong_followed* (at 70B), neither of which has a reactive analogue — is established for the parseable subset, which is sufficient for the existence claim.

9 Causal mechanism tests (Benchmark v3)

The 4-way decomposition of the previous section is correlational: it sorts IM-HYP cells into bins by what the model happens to emit, but it does not perturb the prompt to test whether each bin's harm has the predicted causal source. Benchmark v3 introduces four new prompting arms

designed to perturb exactly one factor at a time and so isolate the mechanism. The arms reuse v2's deterministic task generator with the same `--task-seed=2026`, so v3 cells join task-for-task seed-for-seed with v2.

The four mechanism arms.

im-Hyp-SC$_5$: same prompt as IM-HYP, but at each step we sample 5 hypotheses at $T = 0.7$ with distinct seeds, parse the 5 integers, and emit the majority vote. If *commitment to a single wrong hypothesis* is the cause, this should pull error toward REACTIVE on cells where IM-HYP is in the *wrong_followed* bin — because no single wrong hypothesis dominates the aggregate.

im-Hyp-Oracle: the system prompt is augmented with the true generator rule (e.g., $f(k) = (k-1)^2$) and the model is instructed to apply it. This is a sanity check on whether the model can apply a correct rule when given one. If yes, *rule-application* works and the harm in IM-HYP must be in *rule-generation*.

im-Hyp-First: the prompt is reordered so that the model emits the integer FIRST and the hypothesis on the line below. If the harm is the textual prefix of the hypothesis conditioning the integer-emission step, this should erase the harm.

im-Hyp-EqBudget: same prompt as IM-HYP but with `max_tokens` = 256, matching IM-COT. Closes the budget-asymmetry caveat (Limitation 5 in Section 12).

We run these arms on three model scales. At Qwen-7B and Qwen-14B we run the full grid (30 tasks per band $\times$ 3 seeds, 5 hypothesis samples for SC$_5$). At Qwen-32B, where SC$_5$ would overrun an overnight budget, we run a reduced version with 20 tasks per band, 2 seeds, 3 hypothesis samples, and only the two most-diagnostic arms (IM-HYP-SC$_5$ and IM-HYP-Oracle). Each run also re-runs the v2 baselines (REACTIVE, IM-HYP, IM-COT) on the same task pool so the analysis is self-contained and paired.

9.1 Results

Table 8 reports the mean log-error gap to REACTIVE for every (scale, difficulty, arm) cell. Significance is from paired t-tests on log-error with $n = 90$ paired observations at 7B/14B and $n = 40$ at 32B.

Finding 1 (refutation): self-consistency does not help. IM-HYP-SC$_5$ produces gaps essentially identical to IM-HYP at every (scale, difficulty) cell tested. Across the nine cells tested, the largest gap improvement of SC$_5$ over IM-HYP is +0.31 log-error (at 7B-hard); the smallest is 0.0 (at 14B-easy). None of these improvements approaches REACTIVE. The straightforward tokenized-commitment story — *the model commits to one wrong hypothesis; sample many and aggregate to recover* — is therefore refuted. The natural reading is that the population of sampled hypotheses is correlated in its errors: many sampled hypotheses are wrong in similar ways, and majority-voting their integers preserves the systematic error rather than averaging it out.

Finding 2 (positive deployment result): oracle helps, and the help grows with scale on hard tasks. IM-HYP-Oracle, in which the true rule is given to the model directly, helps significantly at 7/9 cells tested. The two non-significant cells (Qwen-14B-easy and Qwen-32B-easy) are those where REACTIVE is already at the floor (mean error 0.00), leaving no room for any arm to improve. At every cell where REACTIVE has slack, oracle wins. The most striking contrast is on Qwen-32B-hard: REACTIVE scores 20.89 mean asymptotic error, oracle scores 3.38, an 84% reduction ($p = 1.6 \cdot 10^{-3}$). The benefit of oracle on hard tasks *grows monotonically with scale*: 7B $-0.24 \rightarrow$ 14B $-0.25 \rightarrow$ 32B -0.84. The implication for practitioners is direct: *when you can give the LLM the exogenous rule, do so; do not ask it to articulate one.*

Table 8: Causal mechanism tests on benchmark v3. Reported is the mean log-error gap of each arm vs. REACTIVE on the same task pool. Positive entries hurt; negative entries help. Bold marks $p < 0.05$. Three findings: (1) IM-HYP-SC$_5$ is statistically indistinguishable from IM-HYP at every cell tested (self-consistency does not help); (2) IM-HYP-Oracle helps significantly at 7/9 cells, with the help *growing with scale* on hard tasks ($-0.24, -0.25, -0.84$ at 7/14/32B); (3) IM-HYP-EqBudget is numerically identical to IM-HYP (token-budget asymmetry was inert). The arm IM-HYP-First was not run at 32B (overnight-time budget) but at 7B/14B is indistinguishable from or only mildly milder than IM-HYP.

Model	Difficulty	IM-HYP	IM-COT	SC$_5$	Oracle	First	EqBudget
Qwen-7B	easy	**+1.35**†	+0.01	**+1.32**†	**−0.29***	**+1.07**†	**+1.35**†
	medium	**+1.28**†	+0.06	**+1.34**†	**−0.30***	**+1.48**†	**+1.27**†
	hard	**+1.89**†	**+0.17**†	**+1.58**†	**−0.24***	**+1.93**†	**+1.89**†
Qwen-14B	easy	**+1.00**†	**+0.71**†	**+1.00**†	−0.04	**+0.81**†	**+1.00**†
	medium	**+1.54**†	**+0.89**†	**+1.52**†	**−0.14***	**+0.87**†	**+1.54**†
	hard	**+0.99**†	**+1.51**†	**+0.85**†	**−0.25***	**+0.95**†	**+0.99**†
Qwen-32B	easy	**+0.86**†	**+0.22**†	**+0.78**†	−0.07	—	—
	medium	**+1.00**†	**+0.50**†	**+0.91**†	**−0.43***	—	—
	hard	+0.51	**+1.59**†	+0.21	**−0.84***	—	—

Finding 3 (refutation): textual-prefix conditioning is not the mechanism. IM-HYP-First reverses the order so the integer is emitted first; the hypothesis follows on the next line and cannot condition the integer. At 7B all three bands and at 14B all three bands, IM-HYP-First still hurts substantially (+0.81 to +1.93 log-error gap). The harm is not primarily about the autoregressive prefix conditioning the integer.

Finding 4 (cleanup): the budget-asymmetry caveat is inert. IM-HYP-EqBudget gives `max_tokens` = 256 to the IM-Hypothesis prompt, matching IM-COT. At every cell at 7B and 14B the result is numerically identical to IM-HYP within rounding (±0.001 log-error). Limitation 5 of Section 12 is therefore closed.

Revised mechanism story. The four findings together require a sharper account than the original commitment story. The harm of the IM-Hypothesis prompt appears to come from the *rule-generation* step, not from any of (a) commitment to one sampled rule, (b) the textual ordering of rule-then-integer, or (c) the token budget. The model can apply a stated rule reliably (oracle helps); the model cannot reliably generate a correct rule, and once it has emitted some rule, neither resampling nor reordering recovers the implicit pattern-matcher's accuracy. This is consistent with the v2 4-way decomposition (Section 8.1): cells where the parser confirms a correct rule are at the reactive floor at 70B; cells where the parser confirms a wrong rule carry the harm. Benchmark v3 shows that the wrong-rule cells cannot be fixed by sampling diversity (SC$_5$) or by reordering (IM-HYP-First), but they can be eliminated by removing the generation step entirely (Oracle).

Deployment implication. The Oracle result is a genuinely positive finding suitable for practitioner adoption. In settings where the rule generating an exogenous reference is known to the deployer (e.g., a parser is available, the structure is known a priori, the rule can be inferred from documentation), the deployer should put the rule in the system prompt rather than asking the LLM to maintain an internal model. The benefit at Qwen-32B-hard is 84% error reduction; under the rule-generation account, similar benefits should appear at any scale provided the model can parse the stated rule.

10 Cross-paradigm replication: code debugging

The findings reported in Sections 7–9 establish the Apply-Don't-Articulate phenomenon in deterministic integer sequence prediction. A natural reviewer concern is paradigm specificity: *would the same effects appear in a different symbolic-reasoning task?* Code debugging is the natural stress test: it is symbolic and deployment-realistic; the chain-of-thought literature reports its largest gains in this regime [6, 4]; and grading is fully programmatic via hidden test suites, eliminating any LLM-as-judge subjectivity. We apply the v3 condition design verbatim to a code-debugging benchmark and report a clean cross-paradigm replication of every v3 mechanism prediction, with effect sizes that are if anything *larger* than in sequence prediction.

10.1 Benchmark setup

We construct 30 hand-curated buggy Python functions, 10 per difficulty band, paralleling the v1 sequence-prediction benchmark in structure. Each task supplies (i) a canonical (correct) implementation, (ii) a buggy implementation differing from the canonical by a single concrete edit, (iii) a hidden test suite of four to five pass/fail tests including edge cases, and (iv) a ground-truth one-line bug description used by the Oracle arm.

The bands span: **easy** – single-token bugs (off-by-one in `range`, wrong comparison operator, swapped subtraction operands, missing return, etc.); **medium** – control-flow or data-structure bugs (mutable default argument, append-vs-extend, accumulator initialised inside the loop, off-by-one in 2D indexing, etc.); **hard** – semantic bugs (wrong algorithm, wrong sign in distance formula, integer division for float median, off-by-one in binary search, set-loses-order in dedup, etc.). A self-validation pass at module import time confirms that for every template, canonical passes all tests and buggy fails at least one. The full template list is in the public repository.

The grader extracts a Python `def` block from the model's reply (handles markdown fences, prose, multiple defs by selecting the last syntactically-valid candidate, helper functions), exec's it in a restricted-builtins namespace (no `import`, `open`, `exec`), runs the test runner under a SIGALRM wall-clock timeout, and returns a pass-rate in $[0, 1]$. The test runners are in-process callables that are not exposed to the model; the model never sees the test cases.

10.2 Conditions

The seven conditions exactly mirror the v3 design, translated to the code-debugging idiom.

- Reactive: "output the corrected def, no commentary."
- **IM-Diagnosis**: "state the bug as '`# BUG: ...`' first, then the def." Direct analog of IM-Hypothesis.
- **IM-CoT**: "think step by step, then the def."
- **IM-Diag-SC$_5$**: sample 5 diagnoses at $T = 0.7$, group fixes by canonical AST hash, take the modal fix.
- **IM-Oracle**: true bug description supplied in the system prompt; model just applies it.
- **IM-Diag-First**: def first, then '`# BUG: ...`' comment afterward.
- **IM-Diag-EqBudget**: same prompt as IM-Diagnosis with `max_tokens` = 512 matching CoT.

We test on three scales of the Qwen2.5-Coder family — the code-tuned variants, which give the IM prompts the strongest possible test because the model is trained for the task — at 7B, 14B, and 32B. Per-cell sample size is $n = 30$ at 7B and 14B and $n = 20$ at 32B (paired by task and seed).

Table 9: Cross-paradigm replication: mean pass-rate per (scale, difficulty, condition) on the code-debugging benchmark. Bold marks cells where the paired t-test against REACTIVE on the arcsin-square-root-transformed pass-rate is significant at $\alpha = 0.05$ in the hurts direction. Oracle never significantly hurts at any cell; mostly ties or trends positive.

Model	Difficulty	REACTIVE	IM-Diag	CoT	SC_5	Oracle	First	EqBudget
Coder-7B	easy	0.975	0.883	0.917	0.925	1.000	0.925	0.883
	medium	0.808	0.825	0.775	0.908	0.917	0.875	0.825
	hard	0.883	0.850	**0.725**†	0.817	0.967	0.850	0.850
Coder-14B	easy	1.000	**0.033**†	0.917	**0.100**†	1.000	**0.867**†	**0.033**†
	medium	0.975	**0.033**†	0.867	**0.083**†	0.975	**0.742**†	**0.033**†
	hard	0.967	**0.000**†	**0.817**†	**0.033**†	1.000	**0.617**†	**0.000**†
Coder-32B	easy	1.000	**0.800**†	1.000	1.000	1.000	—	—
	medium	0.875	**0.600**†	0.812	**0.650**†	0.975	—	—
	hard	0.975	0.800	**0.800**†	0.950	1.000	—	—

10.3 Results

Table 9 reports the mean pass-rate per (scale, difficulty, condition) cell.

The findings replicate every one of the five v3 mechanism predictions.

Finding 9.1 – Catastrophic harm at 14B-Coder. Asking Qwen2.5-Coder-14B to prepend a single '`# BUG: ...`' comment line collapses pass-rate from $\geq 96\%$ to $\leq 3\%$ at every difficulty band. Paired t-statistics are 25 to 42, p-values $< 10^{-20}$ at every cell. This is the largest single prompt-induced regression observed in the study, and it occurs on a model trained specifically for code generation, applied to an explicitly code-generation task. The 97-percentage-point accuracy drop from a single comment-line addition is, to our knowledge, the strongest documented case of a prompt-induced capability regression in any LLM benchmark.

Finding 9.2 – Inverted-U over scale, replicated. The harm of IM-Diagnosis (relative to REACTIVE) follows the same scale pattern as the v3 inverted-U for hard-band CoT: **barely hurts at 7B-Coder** (only 1 of 18 paired cells significant), **catastrophic at 14B-Coder** (13/18 cells significant; absolute pass-rate near zero), **partial recovery at 32B-Coder** (3 of 9 cells significant; absolute pass-rate 0.6–0.8). The Coder-7B model largely ignores the diagnosis scaffolding because the function-fixing pattern is so dominant in its training; the 14B model follows the scaffolding literally and produces wrong fixes under that constraint; the 32B model partially recovers, presumably by either sidestepping the scaffolding or producing more accurate diagnoses.

Finding 9.3 – Self-consistency does not recover at the catastrophe scale. IM-Diag-SC_5 is statistically indistinguishable from IM-Diagnosis at every 14B cell (absolute pass-rate gap ≤ 0.07 across the three difficulty bands). At 32B, SC_5 recovers fully on easy and hard but not on medium. The v3 finding that "self-consistency over 5 sampled hypotheses does not fix the simple commitment story" replicates verbatim in code: when most sampled diagnoses are wrong, the modal fix is also wrong, and majority voting preserves the systematic error.

Finding 9.4 – Oracle never hurts; trends positive where slack exists. IM-Oracle matches or trends slightly above REACTIVE at every (scale, difficulty) cell tested. At no cell does Oracle significantly hurt at $\alpha = 0.05$. At every Coder-14B and Coder-32B cell where REACTIVE has slack (reactive pass-rate below 1.0), Oracle's pass-rate is at least as high. At

Table 10: Cross-paradigm summary of the v3 mechanism predictions. Each row is a prediction the simple tokenized-commitment account or one of its variants makes; each column is a paradigm in which we tested it. ✓ = prediction supported (i.e., empirical data agrees with the prediction); × = prediction refuted; — = not tested.

Mechanism prediction	v3 Sequence prediction	v3-style Code debugging
SC over 5 samples recovers REACTIVE at the catastrophe scale	× (refuted)	× (refuted)
Oracle (rule given) helps significantly	✓	✓
Reordering (def/integer first) erases the harm	× (refuted; mild relief only)	× (refuted; mild relief only)
Token-budget asymmetry explains the harm	× (refuted; budget inert)	× (refuted; budget inert)
Inverted-U harm peaks at intermediate scale	✓ (peak at 14–32B)	✓ (peak at 14B-Coder)

Qwen2.5-Coder-32B-medium, Oracle delivers +0.10 pass-rate over reactive (0.975 vs. 0.875), and on 32B-hard Oracle matches reactive at 1.000 where reactive is 0.975. The deployment recommendation *provide the rule rather than ask the model to articulate it* transfers cleanly from sequence prediction to code debugging.

Finding 9.5 – Reordering is partially helpful but not sufficient. IM-Diag-First (def first, comment after) hurts substantially at 14B-Coder (Δ pass-rate of +0.13, +0.23, +0.35 on easy/medium/hard relative to IM-Diagnosis) but the harm relative to REACTIVE is still significant at every 14B band. At 7B-Coder the effect is *n.s.* across all three bands. This refines the v3 textual-prefix-conditioning result: at the catastrophe scale, ordering does provide measurable relief, but the harm of articulating-then-emitting is largely order-independent.

Finding 9.6 – Token-budget asymmetry remains inert. IM-Diag-EqBudget produces *numerically identical* cells to IM-Diagnosis at every 14B cell (pass-rate difference 0.000 to within rounding). Limitation 5 of Section 12 is now closed independently in both paradigms.

10.4 Cross-paradigm summary

Table 10 consolidates the v3 sequence-prediction and code-debugging findings. Every one of the five mechanism predictions confirmed in v3 transfers to the code-debugging paradigm.

The Apply-Don't-Articulate phenomenon is now established in two distinct symbolic paradigms: deterministic integer sequence prediction (low-vocabulary, single-token output) and code debugging (high-vocabulary, multi-token output, deployment- realistic). All five mechanism predictions transfer cleanly. The deployment recommendation – *do not ask the LLM to articulate its internal model; provide the model with the rule and ask it to apply it* – is now backed by data from two paradigms, seven distinct model scales, and over 10,000 prompted trials.

11 Discussion

11.1 What the result is and is not

The result is that the prompting analogue of the IMP fails: asking an LLM to externalize an internal model and use it does not improve closed-loop sequence prediction in the regime tested, and reliably worsens it. The result is not that the IMP itself is wrong; it is a theorem about feedback architectures with explicit state, not about prompts. The result also does not imply

that internal models in general are bad; networks plausibly compute something like an internal model in their weights and activations, and that implicit model is what REACTIVE leverages.

11.2 Comparison with the chain-of-thought literature

Wei et al. [6] demonstrate that CoT helps on complex multi-step reasoning, particularly for $\geq$ 100B parameter models. Our largest tested model is 70B-q4 – effectively smaller after quantization – so we are formally below the regime where their strongest gains were observed. Nevertheless, our paradigm (deterministic integer-sequence prediction) is symbolic, the regime where the meta-analysis of Sprague et al. [4] documents the largest CoT gains on average. Our finding – that IM-COT does not help and that the IMP-flavoured IM-HYP hurts substantially across the entire scale range tested – is therefore not contradicted by the original CoT literature but does narrow the regime of CoT applicability further. In particular, the inverted-U harm pattern (Section 7.4) is not predicted by either of the two main CoT papers and is to our knowledge new.

11.3 Implications for treating LLMs as controllers

The take-away for LLM-as-controller programmes is direct: the existence of an implicit internal model in an LLM does not license the prompt-level externalization of that model. In our data, the implicit model embedded in network weights and activations consistently outperforms the model the same network articulates in tokens. This has design implications for agentic systems: prompting an LLM agent to "maintain a world model" or to "state its plan before acting" cannot be assumed to strengthen control performance and may weaken it on deterministic-tracking tasks.

11.4 Implications for prompt engineering

The robustness of the harm under the bootstrap CI (which resamples tasks) refutes the "you got unlucky with task choice" rejoinder. At three of the four tested scales, the medium-band IM-Hypothesis harm is significant under both the parametric paired test and the non-parametric task-cluster bootstrap. This is the level of robustness a careful methodologist requires before reading a negative result as substantive rather than artifactual.

11.5 Connection to multi-agent failure modes

The harm we observe is the single-agent reduction of a class of failures the MAST taxonomy of Cemri et al. [1] catalogues at the system level. Our benchmark gives a controlled probe for the single-agent component of those failures, which can then be composed back to study how an articulated wrong model propagates when consumed by a downstream agent. Pre-registered as future work in Section 13.

12 Limitations

We deliberately use this section to make the scope of our claims precise. Each limitation could in principle reverse the qualitative finding under different choices.

Two symbolic paradigms; non-symbolic paradigms untested. Section 7–9 cover deterministic integer sequence prediction; Section 10 covers Python code debugging. Both are symbolic-reasoning paradigms. Continuous-valued tracking, stochastic prediction, dialog state tracking, multi-step planning, and other non-symbolic deployment regimes are not tested here. The cross-paradigm replication between the two paradigms we do test is consistent with the generalised claim, but does not establish it for non-symbolic tasks.

Open-weight models only; one model family per paradigm. Sequence prediction uses three Qwen2.5 sizes plus one quantized Llama3.1; code debugging uses three Qwen2.5-Coder sizes. No frontier-scale closed model (Claude, GPT, Gemini) is tested in either paradigm; the cross-paradigm consistency we report is within the open-weight regime. Whether the inverted-U peak shifts at frontier scale, or whether Oracle's monotone help saturates, are open empirical questions.

Three prompt phrasings per paradigm. The harm is specific to the exact prompts in Sections 4.1 and 10. Other phrasings of "maintain an internal model" without an explicit `HYPOTHESIS:`/`# BUG:` line, or with explicit self-correction instructions, might give different effects. We have not run robustness sweeps over the prompt-text dimension.

Decoding configuration. v1 uses greedy decoding; v2 and v3 use sampled decoding ($T = 0.7$, 3–5 seeds). Our self-consistency arm uses $N = 5$ samples; whether $N \in \{20, 50\}$ would change the catastrophe-scale result is untested. At $N = 5$ the simple commitment story is refuted in both paradigms; this does not preclude a finer-grained sampling-diversity account that requires more samples than 5 to expose.

Token-budget asymmetry — closed. IM-CoT originally used `max_tokens` = 256 versus 96 for the hypothesis-prefix arms; we flagged this in earlier drafts as a potential confound. The IM-Hyp-EqBudget and IM-Diag-EqBudget arms (Sections 9 and 10) match the budget at 256 and 512 tokens respectively, and produce numerically identical results to their unequal-budget counterparts in both paradigms. This caveat is therefore empirically closed.

13 Future Work

The benchmark suite reported here closes several questions that earlier drafts pre-registered. We organise this section into *closed* (questions empirically resolved by the v2/v3/code benchmarks reported here) and *open* (questions a NeurIPS reviewer is still entitled to ask).

Closed by this paper

C1. Self-consistency over hypotheses. Pre-registered in earlier drafts as F1. Resolved by Section 9 (IM-Hyp-SC$_5$ in sequence prediction) and Section 10 (IM-Diag-SC$_5$ in code debugging). Result in both paradigms: $N = 5$ self-consistency does not recover Reactive at the catastrophe scale; refutes the simple tokenized-commitment account.

C2. Hypothesis-correctness decomposition. Pre-registered as F2. Resolved by Section 8.1.

C3. Reordering / textual-prefix mechanism. Pre-registered as a sub-question of F1. Resolved by IM-Hyp-First and IM-Diag-First arms in Sections 9 and 10. Reordering provides partial relief at the catastrophe scale but does not erase the harm.

C4. Token-budget asymmetry. Pre-registered as Limitation 5. Resolved by IM-Hyp-EqBudget and IM-Diag-EqBudget arms in both paradigms. Budget asymmetry is empirically inert.

C5. Cross-paradigm generality. Pre-registered as F6 (continuous-valued targets) but answered at higher value by Section 10's code-debugging benchmark. All five mechanism predictions transfer cleanly between sequence prediction and code debugging; effect sizes are larger in the code paradigm.

Open

O1. Frontier-model replication. Run both benchmarks on Claude Opus, GPT-5, Gemini, and similar. If the harm disappears at frontier scale, the headline tightens to "below frontier scale, IMP-as-prompt hurts." If it persists in even one of those three, the claim generalises

beyond open-weight models entirely. Straightforward but requires API budget that was out of scope here. Highest priority.

O2. Higher-N self-consistency. Re-run SC arms at $N \in \{20, 50, 100\}$ to test whether the simple commitment story survives at sample sizes where the modal hypothesis becomes more reliably correct. At $N = 5$ the story is refuted; the question is whether a higher-N "sample over many wrong rules until the right one wins" approach can rescue it. We expect not, on the basis that the wrong rules are correlated rather than uniformly random, but this is empirical.

O3. Non-symbolic paradigms. Dialog state tracking, multi-step planning, real-world agent benchmarks (e.g., SWE-bench). The cross-paradigm replication between integer sequence prediction and code debugging is consistent with a general phenomenon, but both paradigms are symbolic. A non-symbolic paradigm where the model has no parseable rule to articulate might not show the harm at all, or might show it for different reasons.

O4. Multi-agent decomposition. In a MAS, the IMP analogue is that one agent maintains an explicit world model and another plays the controller. Running the benchmark with a two-agent (modeller, predictor) split, where the modeller's output is the predictor's prefix, would test whether the failure mode persists across agent boundaries (1).

O5. Architectural intervention. Fine-tune a small (7B) model on data where the integer or fix is emitted *before* the hypothesis or diagnosis text — inverting the standard chain-of-thought ordering. If the fine-tuned model loses the rule-generation harm, that is direct architectural evidence for the cross-paradigm mechanism we propose.

O6. Mechanistic-interpretability probes. Compare the LLM's internal next-token distribution under Reactive to the distribution under IM-Hyp / IM-Diagnosis conditioned on a wrong articulated rule. If the wrong-rule prefix sharply biases the next-token distribution toward outputs consistent with the wrong rule, that is direct activation-level evidence for generation-step commitment. Combined with the Section 9 and 10 behavioural results, this would identify which finer-grained mechanism (correlated sampled-hypothesis errors, generation-step instability) is operative at the activation level.

14 Conclusion

We tested the prompting analogue of the Internal Model Principle in two distinct symbolic-reasoning paradigms: deterministic integer sequence prediction (over 5,760 prompted trajectories across four open-weight scales) and Python code debugging (over 3,000 trials across three Qwen2.5-Coder scales). Across this corpus, four findings stand together.

First, articulating an internal model never significantly helps in either paradigm, at any tested scale, under any of the articulation phrasings tested.

Second, the IM-Hypothesis / IM-Diagnosis prompt significantly hurts in both paradigms. In sequence prediction the harm decays monotonically with scale on the easy band (Spearman $\rho = -1.00$, Pearson $r = -0.998$, $p = 0.002$) but persists on the medium band at every tested scale (pooled Stouffer $p < 10^{-6}$). In code debugging the harm collapses pass-rate from near-1.0 to near-zero at Qwen2.5-Coder-14B at every difficulty band ($t = 25$–42, $p < 10^{-20}$ at every cell) — the largest single prompt-induced regression observed in the entire study.

Third, the harm exhibits a scale-dependent inverted-U in both paradigms, peaking at intermediate scale (14–32B in sequence prediction; 14B-Coder in code debugging) and weakening at the smallest and largest tested scales. The mechanism we propose — *the model articulates competent-looking but wrong rules in tokens, and once articulated, the rule constrains downstream emission* — is consistent with the inverted-U: at small scale articulation is inert, at intermediate scale articulation is articulate-enough to lock in but not accurate-enough to be correct, at large scale articulation is more often correct.

Fourth, four causal-mechanism arms confirm the same revised story in both paradigms. Self-consistency over 5 sampled hypotheses (SC_5) does not recover the reactive baseline at the

catastrophe scale, refuting the simplest commitment account. Reordering (def/integer first) provides partial relief but does not erase the harm, refuting the textual-prefix account. Equalising the token budget is empirically inert, closing the budget caveat. But *providing* the true rule (the Oracle arm) helps significantly with growing magnitude on hard tasks, reaching an 84% error reduction at Qwen-32B-hard in sequence prediction and +0.10 pass-rate at Qwen2.5-Coder-32B-medium in code debugging. The revised mechanism story is: the harm of the IM-Hypothesis / IM-Diagnosis prompt is in the *rule-generation* step — the model can apply a stated rule reliably but cannot reliably write a correct one.

These results yield a clean deployment finding, not just a negative one. In settings where the rule generating an exogenous reference is known to the deployer (a parser is available, the structure is documented, the rule can be inferred from a spec), the deployer should put the rule in the system prompt rather than ask the LLM to maintain an internal model of it. Across two paradigms and seven model scales, *"maintain an internal model" prompts hurt; "apply this stated rule" prompts help.*

Code and data. https://github.com/Minds-R-Lab/phase_margin_llm. Discovery benchmark: `pipeline/experiments/run_imp_benchmark.py`. Confirmation benchmark: `pipeline/experiments/r` Mechanism arms: `pipeline/experiments/run_imp_benchmark3.py`. Code-debugging benchmark: `pipeline/experiments/run_imp_code_benchmark.py` with `pipeline/src/phase_margin/coding/`. Per-cell raw outputs and analysis scripts in `pipeline/results/`.

References

[1] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why Do Multi-Agent LLM Systems Fail? arXiv preprint arXiv:2503.13657, 2025.

[2] B. A. Francis and W. M. Wonham. The internal model principle of control theory. *Automatica*, 12(5):457–465, 1976. doi:10.1016/0005-1098(76)90006-6.

[3] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv preprint arXiv:2205.11916.

[4] Zayne Sprague, Fangcong Yin, Juan Diego Rodriguez, Dongwei Jiang, Manya Wadhwa, Prasann Singhal, Xinyu Zhao, Xi Ye, Kyle Mahowald, and Greg Durrett. To CoT or not to CoT? Chain-of-thought helps mainly on math and symbolic reasoning. arXiv preprint arXiv:2409.12183, 2024. Published at ICLR 2025.

[5] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv preprint arXiv:2203.11171.

[6] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv preprint arXiv:2201.11903.

A Discovery-benchmark task list (v1)

The 30 hand-curated discovery-benchmark tasks (used in Section 4.3) are listed in the public repository under `pipeline/experiments/run_imp_benchmark.py`.

B Confirmation-benchmark task families (v2)

The exact parameter ranges, the set of 260 tasks generated by `--task-seed=2026`, and the family-by-family per-band counts are in `pipeline/experiments/run_imp_benchmark2.py` on the public repository.

C Code-debugging task templates

The 30 buggy-Python-function templates (canonical, buggy, hidden test suite, ground-truth bug description) are defined in `pipeline/src/phase_margin/coding/bug_templates.py`. A self-validation pass at module-import time confirms that for every template, canonical passes all tests and buggy fails at least one.

D Selected raw replies (Qwen-7B, Fibonacci, IM-Hyp)

For illustration of the rule-generation failure mode, the raw replies on `h01_fibonacci` for Qwen-7B under IM-HYP at the last six steps of the trajectory ($k = 8, 9, 10, 11, 12, 13$): true targets are $34, 55, 89, 144, 233, 377$.

```
k=8:   HYPOTHESIS: y_k = y_{k-1} + 8     -> 33
k=9:   HYPOTHESIS: y_k = y_{k-1} + 13    -> 47
k=10:  HYPOTHESIS: y_k = y_{k-1} + 13    -> 68
k=11:  HYPOTHESIS: y_k = y_{k-1} + 21    -> 110
k=12:  HYPOTHESIS: y_k = y_{k-1} + 21    -> 165
k=13:  HYPOTHESIS: y_k = y_{k-1} + 34    -> 267
```

The model articulates a wrong recurrence with a d_k that follows the Fibonacci sequence itself but lagged by one step, and computes the integer prediction deterministically forward from the lagged d_k. Under REACTIVE on the same task the model emits the correct integer at every step. Under IM-HYP-Oracle (the rule given in the system prompt), the same model emits the correct integer at every step. The failure is therefore in *generating* the rule, not in *applying* a known one.